

第 14 周期末项目报告

项目名称: 手机遥控 + turtlesim 迷宫探索 (方向 B)

组员: 위아우자

提交日期: 2026 年 6 月 23 日

一、项目概述

我选了方向 B, 就是用 ROS2 里的 turtlesim 小乌龟做迷宫探索。整个项目的目标是通过手机网页遥控乌龟, 并让它能自动走出迷宫。我把课程里学过的网页开发、Tailscale 组网、ROS2 仿真串成一条完整的链路, 最后实现了手动遥控和自动探索两个模式。

二、系统链路

链路很简单: 手机上的遥控器网页通过 WebSocket 发指令, 指令经过 Tailscale 虚拟局域网传到 WSL 里的桥接程序, 桥接程序再把指令转成 ROS2 的速度消息发给 turtlesim_node, 乌龟就动了。反过来, 乌龟的位置和撞墙状态也会通过同样路径回传到手机页面显示。

讲义里特别强调“单一常驻程序”, 所以我把网络接收和机器人控制写在同一个 Python 程序里, 没有分开两个程序去抢控制权, 这样状态不会乱。

三、环境搭建

先在 WSL 里装了 Tailscale, 登录后拿到虚拟 IP 地址。手机上也装 Tailscale 并登录同一账号。这样手机就能通过那个虚拟 IP 访问 WSL 里跑的网页服务了。

然后启动 ROS2 的 turtlesim_node, 再启动桥接程序。桥接程序默认监听 8080 端口, 手机浏览器打开对应的地址就能看到遥控器界面。中间遇到过 Windows 防火墙挡端口的问题, 后来在防火墙里放行 Tailscale 就好了。

四、迷宫设计

迷宫是在桥接程序里用障碍物列表定义的，每个障碍物用左下角坐标和宽高表示。默认迷宫是 4 乘 4 的完美迷宫，用递归回溯算法生成，已经验证过从起点到终点一定有路。起点在左下角（绿色），终点在右上角（橙色），乌龟半径设了 0.5，碰撞检测时会做膨胀处理，防止乌龟贴着墙蹭过去。

我后来稍微改了几个障碍物的位置，让路径更绕一些，测试自动探索的效果。

五、代码改动

桥接程序本身我基本没大改，主要工作是理解它的碰撞检测和终点判定逻辑。碰撞检测就是预测乌龟下一步的位置会不会和障碍物矩形相交，如果会就把前进速度设成零。终点判定就是算当前位置到终点坐标的距离，小于阈值就算到达。

自动探索的代码写在 `explorer.py` 里，用的是右手法则巡墙器。逻辑是：优先看右边有没有墙，右边没墙就右转；右边有墙但正前方有墙就左转；右边有墙且正前方也没墙就继续前进。这个策略对完美迷宫肯定能走到终点。我把它接进了桥接程序，加了一个“自动/手动”切换按钮，点一下自动按钮，程序就会循环调用探索决策，自动发指令。

网页遥控器我加了方向键的长按支持，按着就持续走，松手就停。同时显示乌龟的实时位置、朝向和是否撞墙的状态，调试很方便。

六、自动探索效果

自动模式跑起来后，乌龟大概十几秒就能从起点走到终点，中间会拐进死路然后退出来，但最终都能到达。我统计了一下，每次通关的碰撞次数和用时都差不多，说明算法比较稳定。转弯的时候我改成了先原地转再前进，避免了同时转弯和前进导致的轨迹偏移问题。

七、遇到的问题

主要问题是手机连不上 WSL 里的网页,折腾了很久才发现是 Windows 防火墙把 Tailscale 的通信挡住了,放行之后就好了。还有一个问题是自动模式下乌龟偶尔会卡在墙角,后来把决策频率调高了一点,并且把转弯和前进两步分开,就解决了。

八、成果与分工

单人完成所有工作: 环境搭建、桥接程序调试、自动探索算法接入、网页界面修改、迷宫调整、测试和报告。演示视频录制了从启动到乌龟自动走到终点的全过程,已提交。所有代码也已上传 [GitHub](#)。

九、总结

做完这个项目,我把整条链路从头到尾跑通了,从手机按键到乌龟动起来,每一步都自己调过。自动探索虽然用的是最简单的右手法则,但能在完美迷宫里稳定到达终点,已经达到了项目要求。如果以后有时间,可以试试 BFS 最短路径或者记录已走格子来减少绕路,不过目前的成果我已经满意了。